STYX

Get paid monthly. The merchant collects no name, email, or card.

SOLANA DEVNET · NOT AUDITED · NO MAINNET DEPLOYMENT

THE PROBLEM

Every payment names the payer.

Card rails keep the record. Public chains publish it.

DEMO

Run it yourself.

```
$ npm install @protocol-01/merchant-sdk @solana/web3.js
$ node merchant-gate.mjs

vaults on chain      : 18
entitlement spread   : ended=9  current=7  paused=2

CURRENT  46d6mEYBrktEFqMkhBjLsEA1rJZDmh2DLzmsafPuDjt7
  the real subscriber  : GRANTED
  a different merchant : DENIED
  a made-up subscriber : DENIED
  is_active on chain   : true   <- the program never sets it false

ENDED  72n5rpWb2qaPSnnzUjbnoWqQ7qJkESWrA3MQbN3K1TZ
  the real subscriber  : DENIED
  is_active on chain   : true
```

demo/merchant-gate.mjs, ABRIDGED · NO KEY, NO SOL, NO ACCOUNT · 273 ms ON 13 AUGUST
THIS PROVES ENTITLEMENT, NOT PRIVACY · DEVNET MOVES: READ THE ADDRESSES OFF THE SCREEN, NEVER OFF THIS PLATE

WHERE THE MARKET STOPS

Processors keep the customer file. Public chains publish it. Mixers hide one payment, not a schedule.

Nobody ships the recurrence.

WHAT WE BRING

The schedule lives inside the private layer, not beside it.

A prepaid vault pays the merchant. No customer file exists to lose.

TODAY

One npm install. The merchant keeps their checkout. We take 1.3%.

Live on devnet, not audited.

THE ASK

One pilot merchant. One funded audit.

Break it first: github.com/IsSlashy/Protocol-01

APPENDIX

Ten plates, none of them in the three minutes.

The three minutes above carry seven plates and 113 words on screen. Everything a jury normally asks for is here instead: the landscape, the use cases, the measured state, the verifier's three answers, the fee schedule, what is in flight, the team, two lists of limits, and the spoken script with its budget. Open the one you are asked about.

APPENDIX A1 · THE LANDSCAPE

Three ways this is solved today, and what each one gives up.

Approaches, not product names. A competitor fact we cannot check on disk is a fact a judge can break in the next question.

APPROACH	WHAT IT SOLVES	WHAT IT GIVES UP
Custodial processor	Hides the payment from the chain, handles recurrence well.	Keeps the identity. The merchant outsources the customer list, and inherits it back in a breach.
Public on-chain subscription	Removes the middleman and the chargeback.	Publishes the payer. Vault fields, retailer and schedule are enumerable by anyone with an RPC endpoint.
Shielded pool, spend side only	Breaks the trace at withdrawal.	Leaves the deposit hop naming a wallet, and handles one payment, not a schedule.
Styx	Relays the deposit and addresses the subscription by a commitment to a secret, so no address can be re-derived to ask whether a wallet subscribes.	Devnet only, not audited, and the crowd a buyer hides in has an effective size of one today. See A8.

APPENDIX A2 · WHO BUYS THIS

Two buyers, and the honest split between running and designed.

RUNNING	**Subscription seller.** A merchant registers a service and reads entitlement through `@protocol-01/merchant-sdk`, published on the public npm registry at 0.1.3. No key, no account, 273 ms.
RUNNING	**One-shot purchase by claim code.** 32 bytes from a CSPRNG, consumed by an atomic increment so two concurrent redeemers get exactly one note, and no expiry since commit `636aa139`. Someone who paid can come back and use it whenever they want.
RUNNING	**Deposit with the buyer off the transaction.** Relayed deposit at leaf 72, slot 486 353 558, re-read by RPC with the buyer absent from the account keys. That is absence from one transaction, not unlinkability: the payment that funded it, one hop earlier, names the wallet.
RUNNING	**An adversarial harness anyone can run.** `verify/p01-verify.mjs`, ten probes, no install and no RPC: three committed fixtures replay offline in about a second, and CI gates all three. It reports our own v3 commitment leak as FAIL, on purpose. A checker that only ever says green has checked nothing.
DESIGNED	**Unlinked spend.** The C7 spend instruction `unshield_denominated_stark_v4` is written and verified against the built binary. It is not deployed. Until it is, see the linkage limit in A8.
NOT CLAIMED	No user count, no revenue, no volume, no waitlist, no partnership. Test value assets only.

APPENDIX A3 · THE MEASURED STATE

Eight measurements, each with the date it was taken and the caveat it carries.

WHAT	MEASURED	WHEN, AND THE CAVEAT
Programs live	10 of 14	Executable at their declared address. The 4 that resolve to null are `p01_zkspl`, `subscription`, `stream` and `whitelist`: the recurring path that does run lives inside `zk_shielded`, so the program named subscription is superseded, not broken. Slot 486 742 009, 22 August.
Tests	4,216 pass, 0 fail	turbo 3,020 over 16 packages, web pool 504, web ui 483, Rust 209. At master `047cd9b2`, 22 August. Higher now after the b7 merge, and we will not guess by how much.
Three test suites, not one	test, test:pool, test:ui	CI invokes all three explicitly, since 22 August. A bare `turbo run test` green proves only the first.
C5 drain	10 tests, CI runs 8	Closed 18 August, in CI since 22 August. The 2 skipped read the built `.so` and the CI job has no SBF toolchain, so CI proves a source property, not the binary's.
Shipped prover	229,640 bytes, sha `51a947e3`	Pinned by `shippedBlob.test.ts`, which also refuses the 192,732-byte pre-coset build (sha `4ace8913`) that the chain rejects. Pinned 22 August.
Pool	79 deposited, 34 spent, 45 unspent	1 SOL pool, re-read 28 August. The closed 0.1 SOL pool holds 82 leaves and 53 unspent — more notes than the open one. The 41/12 printed here before was read mid-campaign on 21 August and never refreshed.
Cost of the relayed journey	0.000485 SOL	Net network fees on the leaf 72 journey, 22 August. No on-disk log reproduces this one. The buyer's total that day was 1.013 SOL: 1.000 note, 0.003 on-chain shield fee, 0.010 operator fee.
Purchase to running subscription	167.76 s	One recorded run, leaf 33, SubscribePrivateStark at 36,127 of 200,000 CU. **Not reproducible from a tag:** the branch and tag named for that freeze are absent from this repo.

APPENDIX A4 · THE VERIFIER'S THREE ANSWERS

It accepted one proof and refused the other two for two different reasons.

A verifier that only ever says yes has proved nothing. The gap between the two refusals is the result.

INPUT	ANSWER	COST	WHAT IT DEMONSTRATES
Honest proof	ACCEPTED	809,812 CU	The deployed verifier accepts what the shipped prover produces.
Forged by one byte	REJECTED	542,150 CU	`InvalidProof` 6003, caught deep in the DEEP and FRI work. A forgery consistent with its own transcript costs a full pass to catch.
Public input tampered	REJECTED	19,777 CU	OOD z mismatch, right after step 1. Breaking Fiat-Shamir dies immediately. A 27× gap between the two refusal paths.

> Two caveats we volunteer. The three figures were measured on devnet on 4 August. The accepted proof is transaction `2sLVyzPW…jZiBR`; the two rejections have no on-disk transaction log, only the README line. And no proof has ever been produced from the browser extension or the phone against the deployed verifier: those two surfaces were verified by hash equality with the shipped blob, which is weaker.

One percent is the operator's cut, and it is not the only fee in the repo.

A judge reading the source finds four fee constants, so here is the whole schedule. Any single figure answer of the form "our fee is one percent" is incomplete on purpose or by accident.

FEE	RATE	WHERE
Operator, on a relayed deposit	1%	`OPERATOR_FEE_BPS = 100n`, enforced server side on the relay route.
Protocol shield, on the way in	0.3%	`SHIELD_FEE_BPS = 30`, on chain, independent of the operator cut.
Protocol unshield, on the way out	0.5%	`UNSHIELD_FEE_BPS = 50`, same file.
Fee splitter default	0.5%	`DEFAULT_FEE_BPS = 50`, ceiling 5%.
Total today, 1 SOL in	1.3%	1.003 SOL to the till, 0.010 SOL to the fee sink.

> Why the pool is shared and not per merchant: the crowd is the product, and it is the only asset that grows for free when a merchant joins. One pool per merchant destroys the thesis. The honest half of that sentence is in A8.
> **What kind of guarantee that is, stated precisely.** We do not encrypt the withdrawal. We make it indistinguishable from the others in the pool, so the guarantee is k-anonymity and it is worth $\log_2(k)$ bits — zero at k = 1. That is an information-theoretic property, not a computational one: there is no key an adversary could later find, and nothing here is broken by a quantum computer. It is also the only privacy property we know of that *improves for free* as the product is used. The cost is that it is worthless on day one and has to be bootstrapped.

Two consequences we designed around. First, a set does not add across denominations, it splits — hence one denomination, 1 SOL, founder decision 21 August (`denominatedPool.ts:215`). The measured cost of getting that wrong is on this page already: the closed 0.1 SOL pool holds 53 unspent notes and the open 1 SOL pool holds 45, and those are two crowds of 53 and 45, never one of 98. Second, k is set by the *coarsest side channel*, not by the leaf count: any channel that partitions the pool — the fee payer, the deposit funder, timing — divides k, and one open channel pins it to 1 no matter how many notes are in the tree. That is why the number on the plate is one and not 47, and why A8 lists the channels rather than the leaves.

APPENDIX A6 · WHAT IS IN FLIGHT

C7 does the work of two proofs, is measured, and is not deployed.

It is on a plate because it is already measurable, not because it is scheduled. It merges the commitment derivation and the Merkle circuits so the note commitment never leaves the circuit as a public input.

MEASUREMENT	C7	WHAT IT REPLACES
Verifier cost, phase 1	878,756 CU	1,681,540 CU for the C1 and C3 pair, phase 1 only.
Verifier cost, phase 2	192,715 CU	·
Proof on the wire	77,965 bytes	147,038 bytes for C1 plus C3 today: C1 at 27 queries, C3 at 22, both at ffps 16. A 1.9× cut, not the 3.3× some comments in this repo still claim from the pre-B4 sizes.
What the buyer actually pushes	Not measured for C7	A proof does not fit in one transaction: the deployed C1 plus C3 pair uploads in 74 to 145 `write_proof_chunk` transactions. Bytes are the unit we optimised. Transactions and seconds are the unit the buyer pays.
What it costs everything else	+3,232 to +3,930 CU	Phase 1 on C1 through C6. C0 went down 16 CU. Phase 2 moved by at most one unit.

- **Not deployed.** Extend the verifier account *and* the pool account, which has zero spare room, then push the binary, then rebuild the WASM client. The client rebuild is deliberately blocked until the on-chain verifier answers, because a client that proves what the chain will not accept fails at the end of a 150 transaction upload.
- Put the compute unit harness back in CI. It was removed and nothing re-measures automatically today.
- External audit and mainnet are both unchecked on the public roadmap. They are listed, not scheduled.

APPENDIX A7 · TEAM

The repository names one author, and we will not assert more than that.

- One human author on every commit in this repository, plus a deploy bot. That is the whole of what the history supports.
- Where the published packages carry an author field it reads "Protocol 01" or "Protocol 01 Team", and four carry none at all. That is a label chosen for npm, not a roster.
- The project renamed from Protocol 01 to Styx. The npm scope, the repository and about a hundred on-chain string literals still read `protocol-01`, which is why the demo plate installs `@protocol-01/merchant-sdk`. Those literals are frozen: renaming them moves funds to addresses nobody controls.
- Credentials and employers are not recorded in the repo, so they are not asserted here. Ask the presenter and get the answer from a person.

> This plate exists so that a jury asking about the team gets a straight answer in one line, instead of a slide of logos that has to be walked back.

The list a hostile reviewer would write, written first, by us.

- **Devnet only.** No mainnet deployment. Test value assets only, on Solana devnet and Starknet Sepolia.
- **Not audited.** No external security audit has been performed.
- **Deposit to withdraw linkage is possible on four of the seven spends.** The v3 withdrawal, the v3 subscription, the transfer and the split all republish the commitment their deposit already published — at instruction byte 80, 160, 80 and 80 respectively — so matching the two is trivial for an observer. The crowd of 45 unspent notes is a ceiling; the effective size today is one. C7 closes it for the withdrawal AND, since 27 August, for the subscription — both proven end to end on devnet, both publishing no commitment on the wire. Two of the three clients route to them: this web app and the extension. The phone still calls the v3 path, and the reason is not the one this plate used to give: the "180 s on-device proving" figure was retracted the evening it was recorded (the real device number is C3 = 1,482 ms; the 180 s was a WebView hang). The actual blocker was upstream — the phone's WebView bridge had no spend entry point at all until 27 August, so it could not produce a circuit-7 proof to time.
- **A spend can be walked back to a wallet — on every path but one.** An ephemeral key cannot pay a fee from nothing, so on the ordinary path a transfer funds it and another sweeps the residue, and both are public. **Measured 28 August, that stopped being true of one instruction.** `unshield_denominated_stark_v4_relayed` pays its submitter out of the protocol fee the pool already charges, so a stranger can afford to send the transaction and no lamport travels from the buyer to them. On spend `4KHpG7ka…` the buyer's address appears in the account keys of NONE of the 94 transactions the probe read across three surfaces, each read in full — probe P11, PASS, frozen as `verify/fixtures/v4-relayed` and replayed in CI. ⚠ Read the limits with it: this is one spend on devnet, not a product path — there is no button for it in the app, the relaying node is not hosted, and the fee payer of every OTHER spend still has a funding history. The payee also stays written in the clear at instruction byte 115, permanently, so it re-links the moment those funds move.
- **The proof is not a hiding object.** Circuit C0's witness can be recovered from a single proof. Treat a proof as public data. This is why the word "zero-knowledge" is not on any plate.
- **A merchant's subscribers are publicly enumerable.** One filtered `getProgramAccounts` returns every vault with its retailer, deposit, rate and schedule. What the design buys is narrower: no address can be re-derived to ask whether a given wallet subscribes. And two of the twenty-eight live vaults are legacy normal-mode: they name the subscriber's wallet in the clear, so for those two even that narrower claim does not hold. Re-counted on chain 26 August — 28 `SubscriptionVault` accounts, 2 with a `subscriber_pubkey` present; one of those two is the deployment wallet itself.

- **Client surfaces are proven by hash, not by execution.** No proof has ever been produced from the extension or the phone against the deployed verifier.

APPENDIX A9 · THE OPERATIONAL LIST

The five questions a jury asks that are not about cryptography.

- **One key can replace every program.** All eighteen devnet deployments, superseded builds included, are upgradeable by a single key. No multisig, no timelock, none immutable. The deployed verifier's bytecode is also not proven to come from this repository. A multisig upgrade authority is a precondition of real money, not a nice-to-have.
- **A subscription cannot be renewed or cancelled.** The vault is a one-way prepaid envelope: `claim_period` is the only exit, the two cancel instructions were removed, and nothing returns to the subscriber. No renew instruction exists, and the obvious implementation, reusing the note secret, collides the vault address and strands the second note.
- **The chain's own entitlement flag is wrong.** `subscribe_private_stark.rs` writes `is_active = true` and no instruction ever writes false, so a merchant who reads the field instead of the SDK grants access to every expired subscriber. The SDK computes entitlement from the funding and the clock and is correct today. The field is not, and the fix is a program upgrade.
- **The relay operator sees what the chain does not.** The deposit relay keys a rate limit on the client IP, and the site stores waitlist emails. Off-chain metadata is a trust assumption in the operator, not a proof.
- **Rent already spent is not recoverable.** Proof buffer accounts hold rent the current close path cannot reclaim.

APPENDIX A10 · THE SCRIPT

180 seconds, 398 spoken words, 110 on screen.

The deck was cut to a clock, so the clock is on every plate. At 145 words a minute, a plate gets what its badge says and not a word more. The full spoken text lives in `docs/deck/three-minute-script.md`.

PLATE	BUDGET	SPOKEN	ON SCREEN
01 Title	0:10	23 words	12
02 The problem	0:20	48	16
03 The demo	1:00	120	4 plus the transcript
04 Where the market stops	0:25	56	24
05 What we bring	0:25	57	25
06 What a merchant gets	0:25	59	17
07 The ask	0:15	35	12
Slack	0:00	·	15 s left for the demo to breathe

> One claim per plate. What is written under a headline is the evidence the presenter says out loud, not text for the room to read. The moment a plate carries two claims, the room reads the second while you are still saying the first.